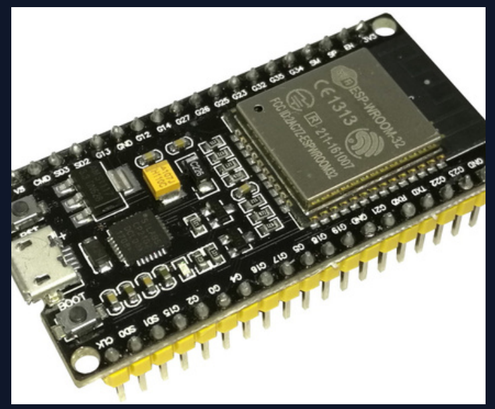
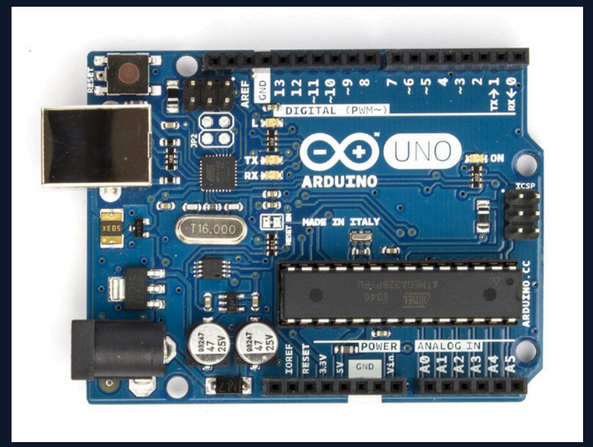
# MicroPython ↔ Arduino UNO

Tres ejemplos gemelos: LED, PWM y servo continuo por serial



ESP32 + MicroPython




Arduino UNO

# Objetivos de aprendizaje

## Entender

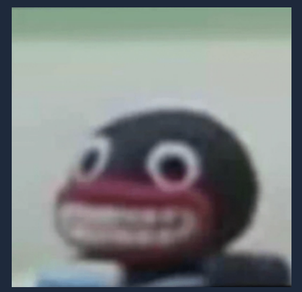
- Qué hace un pin digital.
- Qué hace el PWM.
- Por qué un servo continuo no usa ángulo.

## Comparar

- Pin vs pinMode.
- PWM vs analogWrite.
- input() vs Serial Monitor.

## Aplicar

- Parpadear un LED.
- Crear efecto respiración.
- Controlar velocidad por serial.



# Mismo problema, dos lenguajes

La lógica del microcontrolador se repite: configurar, ejecutar, esperar y repetir.

**1**

### Configurar pin

Entrada, salida, PWM o serial

**2**

### Enviar orden

HIGH/LOW, duty, pulso o texto

**3**

### Esperar

delay/sleep sin perder el estado del hardware

**4**

### Repetir

while True / loop()

**MicroPython: más interactivo, REPL, objetos de hardware.**
**Arduino: estructura fija setup()/loop(), compilación y carga.**

# Ejemplo 1: Blinking LED

Encender y apagar un LED periódicamente.

**MicroPython / ESP32**

```
from machine import Pin
from time import sleep

led = Pin(2, Pin.OUT)

while True:
    led.on()
    sleep(0.5)
    led.off()
    sleep(0.5)
```

**Arduino UNO**

```
const int LED_PIN = 13;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

# Funciones equivalentes: salida digital

Ambos escriben un nivel lógico en un pin, pero la sintaxis cambia.

## MicroPython

| | |
|---|---|
| `Pin(2, Pin.OUT)` | crea un objeto pin como salida |
| `led.on()` | pone el pin en alto |
| `led.off()` | pone el pin en bajo |
| `sleep(0.5)` | pausa 0.5 s |

## Arduino

| | |
|---|---|
| `pinMode(13, OUTPUT)` | configura el pin como salida |
| `digitalWrite(HIGH)` | pone el pin en alto |
| `digitalWrite(LOW)` | pone el pin en bajo |
| `delay(500)` | pausa 500 ms |

**Nota didáctica: en MicroPython se trabaja con objetos de hardware; en Arduino se trabaja con funciones globales y constantes.**



Refs: MicroPython Pin · Arduino digitalWrite

# Ejemplo 2: LED que respira

Sube y baja el brillo con PWM.

**MicroPython / ESP32**

```python
from machine import Pin, PWM
from time import sleep_ms

LED_PIN = 2
pwm = PWM(Pin(LED_PIN), freq=1000, duty_u16=0)

PASO = 512
RETARDO_MS = 15

while True:
    for duty in range(0, 65536, PASO):
        pwm.duty_u16(duty)
        sleep_ms(RETARDO_MS)

    for duty in range(65535, -1, -PASO):
        pwm.duty_u16(duty)
        sleep_ms(RETARDO_MS)
```

**Arduino UNO**

```arduino
const int LED_PWM = 9;
int paso = 5;
int retardoMs = 20;

void setup() {
  pinMode(LED_PWM, OUTPUT);
}

void loop() {
  for (int brillo = 0; brillo <= 255; brillo += paso) {
    analogWrite(LED_PWM, brillo);
    delay(retardoMs);
  }

  for (int brillo = 255; brillo >= 0; brillo -= paso) {
    analogWrite(LED_PWM, brillo);
    delay(retardoMs);
  }
}
```

# PWM vs analogWrite()

PWM no es voltaje analógico real: es encender/apagar muy rápido.

## MicroPython PWM

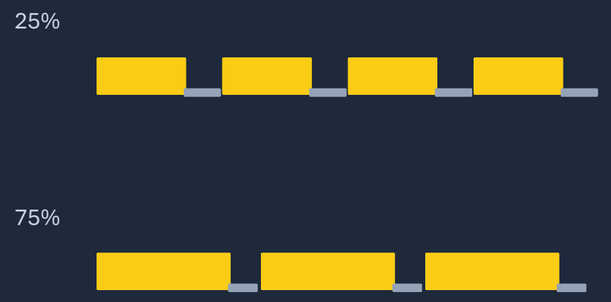
- Objeto explícito: PWM(Pin(...))
- Frecuencia configurable: freq=1000
- Duty de 0 a 65535 con duty_u16()
- También puede usar duty_ns()

## Arduino analogWrite

- Función directa: analogWrite(pin, valor)
- Valor típico en UNO: 0 a 255
- Solo en pines PWM
- Frecuencia depende del pin/timer

## Idea visual

Más tiempo en HIGH = más brillo




En Arduino UNO: PWM en 3, 5, 6, 9, 10, 11.
En ESP32: más flexible, pero depende del pin/firmware.



Refs: machine.PWM · analogWrite · UNO Rev3

# Ejemplo 3: servo continuo por serial

El comando serial define velocidad y sentido, no posición.

## Cableado típico

- Café / negro → GND
- Rojo → 5 V
- Naranja / amarillo → señal PWM

**Rango de control: -100 → 1000 µs | 0 → 1500 µs ≈ stop | 100 → 2000 µs**

Un servo continuo convierte el ancho de pulso en velocidad/sentido. El punto exacto de stop puede requerir calibración fina.

# MicroPython: servo continuo por serial USB

Entrada desde la Shell / REPL de Thonny con input().

**MicroPython / ESP32**

```
from machine import Pin, PWM
from time import sleep_ms

SERVO_PIN = 18

servo = PWM(Pin(SERVO_PIN), freq=50)
servo.duty_ns(1500000)   # stop inicial ~1500 us

print("Escribe un valor entre 1000 y 2000 us")
print("1500 = stop aprox")

while True:
    dato = input("Pulso_us: ")

    pulso_us = int(dato)

    if pulso_us < 1000:
        pulso_us = 1000
    if pulso_us > 2000:
        pulso_us = 2000

    servo.duty_ns(pulso_us * 1000)
    print("Pulso aplicado:", pulso_us, "us")
    sleep_ms(50)
```

## Funciones clave

- PWM(Pin(...), freq=50)
- duty_ns(1500000)
- input() lee texto del serial USB

# Arduino UNO: servo continuo

Entrada desde Serial Monitor con Serial.parseInt().

**Arduino UNO**

```
#include <Servo.h>

Servo servoMotor;

const int SERVO_PIN = 9;

void setup() {
  Serial.begin(9600);

  servoMotor.attach(SERVO_PIN);
  servoMotor.writeMicroseconds(1500);  // stop inicial ~1500 us

  Serial.println("Escribe un valor entre 1000 y 2000 us");
  Serial.println("1500 = stop aprox");
}

void loop() {
  if (Serial.available() > 0) {
    int pulso_us = Serial.parseInt();

    if (pulso_us < 1000) {
      pulso_us = 1000;
    }
    if (pulso_us > 2000) {
      pulso_us = 2000;
    }

    servoMotor.writeMicroseconds(pulso_us);

    Serial.print("Pulso aplicado: ");
    Serial.print(pulso_us);
    Serial.println(" us");

    delay(50);

    while (Serial.available() > 0) {
      Serial.read();
    }
  }
}
```

## Funciones clave

- #include <Servo.h>
- Servo servoMotor;
- attach(pin) conecta la señal
- writeMicroseconds(us) manda el pulso
- Serial.available() revisa si llegó dato
- Serial.parseInt() lee el número

# Serial comparado: REPL vs Serial Monitor

Ambos envían texto por USB, pero el flujo de trabajo es distinto.

## MicroPython / Thonny

- El REPL ya está vivo en la placa.
- input() puede leer desde la consola.
- Puedes probar líneas sin recompilar.
- Ideal para exploración rápida.

## Arduino IDE

- El programa se compila y carga.
- Serial.begin(9600) abre el canal.
- Serial Monitor manda y recibe texto.
- parseInt() interpreta números.

**La diferencia práctica: MicroPython favorece probar y modificar en vivo; Arduino favorece el ciclo compilar → cargar → observar.**

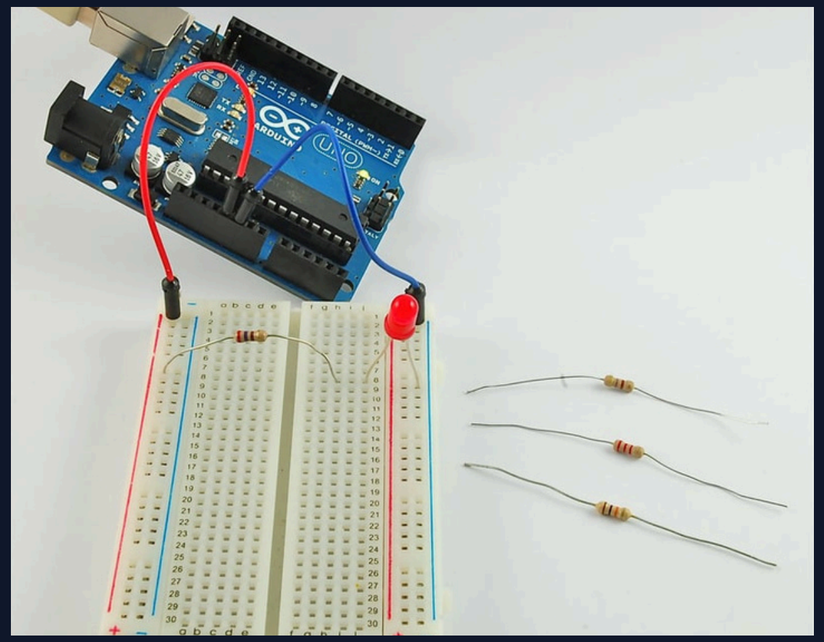
# Tabla de equivalencias rápidas

Una chuleta para pasar de un ecosistema al otro.

| Concepto | MicroPython | Arduino UNO |
|---|---|---|
| Inicio del programa | `script directo / main.py` | `setup() + loop()` |
| Pin de salida | `Pin(2, Pin.OUT)` | `pinMode(13, OUTPUT)` |
| Encender LED | `led.on() / value(1)` | `digitalWrite(pin, HIGH)` |
| Apagar LED | `led.off() / value(0)` | `digitalWrite(pin, LOW)` |
| Pausa | `sleep() / sleep_ms()` | `delay()` |
| PWM LED | `PWM(Pin(...)).duty_u16()` | `analogWrite(pin, 0..255)` |
| Servo continuo | `PWM 50 Hz + duty_ns()` | `Servo.h` |
| Serial USB | `input() en REPL / Shell` | `Serial Monitor + parseInt()` |

# Buenas prácticas de laboratorio

Evita errores comunes al pasar del código al hardware real.



- Usa resistencia con LED externo.
- No todos los pines sirven para PWM.
- Comparte GND entre fuentes externas y la placa.
- No alimentes servos grandes desde el pin de 3.3 V.

**Criterio de diagnóstico** **Si el código “está bien” pero el hardware no responde, revisa primero: pin correcto, tierra común, alimentación, puerto serial.**